

WEEK 3b

EL 162 / 234 OBJECT ORIENTED PROGRAMMING

By: Dr. Matthew Cobbinah

Objectives

At the end of this lecture, students should be able to:

- Work with Special Methods as one of the goals of OOP
 - ✓ Hide implementation details and expose a simple interface.

Special Methods (Magic/Dunder)

SPECIAL METHODS

What are Special Methods?

- Predefined methods that begin and end with **double underscores**.
- They are also called **magic** or **dunder** (short for **Double UNDERscore**) methods
- They allow Python objects to respond to **built-in** operations in simple ways
- Special methods are not a separate method type.
 - ✓ They can be instance methods, class-oriented methods, or methods that operate before an instance exists.
 - ✓ What makes them "special" is not their parameter list, but that Python automatically invokes them in response to language syntax and built-in operations.

For example:

- `len(obj)` internally calls `obj.__len__()`
- `obj1 == obj2` calls `obj1.__eq__(obj2)`

SPECIAL METHODS...

- Special methods allow your objects to behave like built-in types.
- So special methods are not duplicates of built-in functions.
- They are the mechanism by which **custom objects** teach Python **how to apply built-in** operations to them.
 - That is why they are fundamental to making user-defined classes feel like native Python types.

Example:

- Python already has the + operator, so 3+7, will give you 10
- However, if these values are object data, you would have to teach Python how it should apply the add (+) inbuilt method on the objects
- So you define:

```
def __add__(self, other):  
    return addnumbers(self.items + other.items)
```

- Now Python can interpret:

```
num1 + num2
```

Common Examples of Special Methods

1. Constructor `__init__()`
2. string representation `__str__()`, `__repr__()`
3. length `__len__()`
4. equality `__eq__()`
5. operator overloading `__add__()`, `__sub__()`, `__mul__()`
6. item access `__getitem__()`
7. item assignment `__setitem__()`
8. membership testing `__contains__()`
9. callable objects `__call__()`
10. iteration support `__iter__()`, `__next__()`
11. Boolean evaluation `__bool__()`
12. Destructor `__del__()`

1. Constructors in Python (`__init__`)

- A constructor is a special method in Python called automatically ***every time a class is being used to create a new object.***
- It is used to initialize the attributes of an object.
- Without a constructor, every attribute would have to be assigned manually after object creation, which is inefficient and error-prone.

Key Points

- ✓ Named `__init__`
- ✓ Runs automatically during object creation
- ✓ Uses `self` to refer to the current object
- ✓ Ensures objects start in a valid state

Purpose

- ✓ Assign initial values to object attributes
- ✓ Prepare the object for use immediately after creation

1. Constructors in Python (__init__)...

- **Constructor Example 1:**

```
class Student:
    def __init__(self, name, age):
        self.name = name
        self.age = age

s1 = Student("Matthew", 30)

print(s1.name)
print(s1.age)
```

*When Python executes **Student("Matthew", 30)**, it automatically calls, **s1.__init__("Matthew", 30)***

1. Constructors in Python (__init__)...

- **Constructor Example 2:**

```
class Book:
    def __init__(self, title, author):
        self.title = title
        self.author = author

    def describe(self):
        print(f"{self.title} by {self.author}")

b1 = Book("OOP", "Dr. Cobbinah")
b1.describe()
```

2. String Representation Methods

- Python provides two major string representation methods.
 - ✓ **Informal Representation:** `__str__()`
 - ✓ **Formal Representation:** `__repr__()`

2.1 Informal Representation: `__str__()`

- The `__str__()` method provides a human-readable representation of an object.
- It defines what should be displayed when an object is printed.
- Without this method, Python displays the object's memory location, which is not useful to users.
- This method is intended for end users rather than developers.

Syntax

```
def __str__(self):  
    return string
```

2.1 Informal Representation: `__str__()`

Example:

```
class Student:
    def __init__(self, name):
        self.name = name

    def __str__(self):
        return f"Student: {self.name}"

s = Student("Matthew")
print(s)
```

#Output

Student: Matthew

`__str__()` *controls how users see an object.*

2.2 Formal String Representation (`__repr__()`)

- The `__repr__()` method provides an official or developer-oriented representation of an object.
- It is primarily used for debugging and logging.
- Ideally, the returned string should contain enough information to recreate the object.
- When `__str__()` is not defined, Python falls back to `__repr__()`.

Syntax

```
def __repr__(self):  
    return string
```

2.2 Formal String Representation (`__repr__()`)

Example

```
class Student:
    def __init__(self, name):
        self.name = name

    def __repr__(self):
        return f"Student('{self.name}')"

s = Student("Daniella")
print(repr(s))
```

#Output

```
Student('Daniella')
```

`__repr__()` is aimed at developers; `__str__()` is aimed at users.

Difference Between `__str__()` and `__repr__()`

```
class Student:
    def __init__(self, name):
        self.name = name

    def __str__(self):
        return f"Name: {self.name}"

    def __repr__(self):
        return f"Student('{self.name}')
```

```
s = Student("Matthew")
print(s)
print(repr(s))
```

#Output:

Name: Matthew

Student('Matthew')

3. Length Method: `__len__()`

- The `__len__()` method allows custom objects to define what their length means.
- Python's built-in collections such as lists, tuples, dictionaries, and strings all implement this method.
- By implementing `__len__()`, custom objects can work naturally with the built-in `len()` function.

Syntax

```
def __len__(self):  
    return integer
```


3. Length Method: `__len__()`

Example:

```
class Playlist:
    def __init__(self, songs):
        self.songs = songs

    def __len__(self):
        return len(self.songs)

p = Playlist(["Song1", "Song2", "Song3"])

print(len(p))
```

A call to **`len(p)`** means Python internally calls the **`p.__len__()`**

4. Equality Comparison: `__eq__()`

- The `__eq__()` method defines how objects should be compared for equality.
- By default, Python compares whether two variables refer to the same memory location.
- Many applications, however, require comparison based on object content rather than identity.
- When we call `object1 == object2`, internally, Python does this `object1.__eq__(object2)`

Syntax

```
def __eq__(self, other):  
    return boolean
```

4. Equality Comparison: `__eq__()`

Example

```
class Student:
    def __init__(self, student_id):
        self.student_id = student_id

    def __eq__(self, other):
        return self.student_id == other.student_id

s1 = Student(101)
s2 = Student(101)

print(s1 == s2)
```

Without `__eq__`, Python would compare memory addresses instead.

5. Operator Overloading

- **Operator overloading** allows programmers to define how operators behave when applied to custom objects.
- Allows operators to work with custom objects.
- This enables objects to behave naturally and intuitively.
- For example, adding two vectors should produce another vector rather than raising an error.

When is it called?

```
obj1 + obj2
```

Internally:

```
obj1.__add__(obj2)
```

Syntax

```
def __add__(self, other):  
    ...
```

5.1 Addition: `__add__()`

Example

```
class Vector:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __add__(self, other):
        return Vector(
            self.x + other.x,
            self.y + other.y
        )

    def __str__(self):
        return f"({self.x}, {self.y})"

v1 = Vector(2, 3)
v2 = Vector(4, 5)

print(v1 + v2)
```

5.2 Subtraction: `__sub__()`

```
class Vector:
    def __init__(self, x):
        self.x = x

    def __sub__(self, other):
        return Vector(self.x - other.x)

    def __str__(self):
        return str(self.x)
```

```
v1 = Vector(10)
```

```
v2 = Vector(3)
```

```
print(v1 - v2)
```

#Output:

7

5.3 Multiplication: `__mul__()`

```
class Number:
    def __init__(self, value):
        self.value = value

    def __mul__(self, other):
        return Number(self.value * other.value)

    def __str__(self):
        return str(self.value)
```

```
n1 = Number(5)
```

```
n2 = Number(4)
```

```
print(n1 * n2)
```

#Output:

20

Common Operator Overloading Methods

Operator	Method
+	<code>__add__</code>
-	<code>__sub__</code>
*	<code>__mul__</code>
/	<code>__truediv__</code>
//	<code>__floordiv__</code>
%	<code>__mod__</code>
**	<code>__pow__</code>
==	<code>__eq__</code>
!=	<code>__ne__</code>
<	<code>__lt__</code>
<=	<code>__le__</code>
>	<code>__gt__</code>
>=	<code>__ge__</code>

6. Item Access: `__getitem__()`

- The `__getitem__()` method enables an **object** to behave like a sequence or container.
- It allows elements to be accessed using indexing notation, just like lists and tuples.
- Implementing this method makes custom objects more intuitive and compatible with Python's indexing syntax.

Syntax

```
def __getitem__(self, index):  
    ...
```

6. Item Access: `__getitem__()`

Example

```
class Grades:
    def __init__(self):
        self.data = [85, 90, 78]

    def __getitem__(self, index):
        return self.data[index]

g = Grades()

print(g[0])
print(g[1])
```

Python internally executes: `g.__getitem__(0)`

7. Item Assignment: `__setitem__()`

- The `__setitem__()` method enables assignment using indexing notation.
 - ✓ It allows assignment using indexing.
- It allows objects to store or modify values in a way similar to lists and dictionaries.
- This method is commonly used when building custom container classes.

When is it called?

```
object[index] = value
```

Internally:

```
object.__setitem__(index, value)
```

7. Item Assignment: `__setitem__()`

Example

```
class Grades:
    def __init__(self):
        self.data = [85, 90, 78]

    def __setitem__(self, index, value):
        self.data[index] = value
```

```
g = Grades()
```

```
g[1] = 100
```

```
print(g.data)
```

#Output: [85, 100, 78]

8. Membership Testing: `__contains__()`

- The `__contains__()` method defines membership testing for custom objects.
- It determines how the `in` operator behaves.
- This is useful when an object manages a collection of elements and should support membership queries

When is it called?

`item in object`

Internally: `object.__contains__(item)`

8. Membership Testing: `__contains__()`...

Example

```
class Course:
    def __init__(self):
        self.students = ["Ella", "Sandra", "Joe"]

    def __contains__(self, student):
        return student in self.students
```

```
c = Course()
print("Joe" in c)
print("Gifty" in c)
```

#Output:

```
True
False
```

9. Callable Objects: `__call__()`

- The `__call__()` method allows an object to be invoked (or behave) like a function.
- This is useful when objects need to maintain internal state while still behaving like callable functions.
- Many advanced Python frameworks use callable objects extensively.

When is it called?

```
object(arguments)
```

Internally:

```
object.__call__(arguments)
```

9. Callable Objects: `__call__()`

Example

```
class Multiplier:
    def __init__(self, factor):
        self.factor = factor

    def __call__(self, number):
        return number * self.factor

double = Multiplier(2)
print(double(10))      #double now behaves like a function
```

#Output:

```
20
```


10. Iteration Support: `__iter__()` and `__next__()`

- These methods allow objects to work in loops/iteration.
- Together they form the foundation of Python's iterator protocol, which powers for loops.
- An iterator must know how to provide the next value and when iteration should stop.

When are they called?

```
for item in object:  
    ...
```

Python repeatedly calls:

```
__iter__()  
__next__()
```

until a `StopIteration` exception is raised.

10. Iteration Support: `__iter__()` and `__next__()`

Example

```
class Counter:
    def __init__(self, limit):
        self.limit = limit
        self.current = 1

    def __iter__(self):
        return self

    def __next__(self):
        if self.current > self.limit:
            raise StopIteration
        value = self.current
        self.current += 1
        return value

for num in Counter(5):
    print(num)
```

Output:

1
2
3
4

11. Boolean Evaluation: `__bool__()`

- The `__bool__()` method determines how an object behaves in Boolean contexts.
- It basically controls truth-value testing.
- It allows objects to define what should be considered True or False.
- This method is commonly used in classes representing states, conditions, or resources.

When is it called?

```
if object:  
  
or  
  
bool(object)
```

11. Boolean Evaluation: `__bool__()`

Example

```
class BankAccount:
    def __init__(self, balance):
        self.balance = balance

    def __bool__(self):
        return self.balance > 0

account = BankAccount(500)

if account:
    print("Account is active")
```

#Output:

```
Account is active
```

12. Destructor Method: `__del__()`

- The `__del__()` method is known as the **destructor**.
- It is called/executed when an object is about to be removed from memory or when an object is being destroyed.
- Historically it was used for resource cleanup such as closing files or releasing network connections.
- However, because Python's garbage collection timing is not always predictable, modern Python programs generally prefer context managers (with statements) for cleanup tasks.
 - ✓ Avoid relying on `__del__()` for critical cleanup operations.

Syntax

```
def __del__(self):  
    ...
```

12. Destructor Method: `__del__()`

Example 1:

```
class FileHandler:

    def __init__(self, filename):
        self.filename = filename
        print("File opened")

    def __del__(self):
        print("File closed")

f = FileHandler("data.txt")
```

12. Destructor Method: `__del__()`

Example 2

```
class Resource:
    def __init__(self):
        print("Resource allocated")

    def __del__(self):
        print("Resource released")

r = Resource()
```

Special Methods in Summary

Special Method	Purpose
<code>__init__</code>	Initialize object
<code>__str__</code>	User-friendly string
<code>__repr__</code>	Developer/debug representation
<code>__len__</code>	Define object length
<code>__eq__</code>	Equality comparison
<code>__lt__</code> , <code>__gt__</code> , etc.	Relational comparisons
<code>__add__</code> , <code>__sub__</code> , <code>__mul__</code>	Operator overloading
<code>__getitem__</code>	Index access
<code>__setitem__</code>	Index assignment
<code>__contains__</code>	Membership testing
<code>__iter__</code>	Create iterator
<code>__next__</code>	Return next item
<code>__call__</code>	Make object callable
<code>__bool__</code>	Truth-value testing
<code>__del__</code>	Destructor

LAB Session

WEEK 3b - Complete the ShoppingCart class by implementing the required special methods.

```
class ShoppingCart:
    def __init__(self, items):
        pass

    def __str__(self):
        pass

    def __len__(self):
        pass

    def __getitem__(self, index):
        pass

    def __setitem__(self, index, value):
        pass

    def __eq__(self, other):
        pass

    # Challenge
    def __add__(self, other):
        pass
```

```
# Test Code
cart1 = ShoppingCart(["Laptop", "Mouse", "Keyboard"])
cart2 = ShoppingCart(["Laptop", "Mouse", "Keyboard"])

print(cart1)      # String representation
print(len(cart1)) # Number of items
print(cart1[0])   # Access first item

cart1[1] = "Headset" # Modify second item

print(cart1)
print(cart1 == cart2) # Compare carts

# Challenge
cart3 = cart1 + cart2
print(cart3)
```

Expected Output

```
Shopping Cart: ['Laptop', 'Mouse', 'Keyboard']
3
Laptop
Shopping Cart: ['Laptop', 'Headset', 'Keyboard']
False
Shopping Cart: ['Laptop', 'Headset', 'Keyboard',
                'Laptop', 'Mouse', 'Keyboard']
```

Week 3b — Key Takeaways

1. Python Special Methods

2. Cleaner code

3. Integration with built-in functions

4. Custom behavior for user-defined types

5. Operator overloading

6. ShoppingCart class with special methods